

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

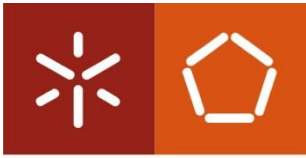
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

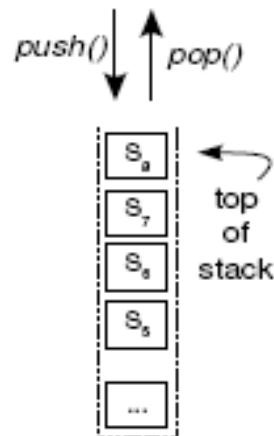
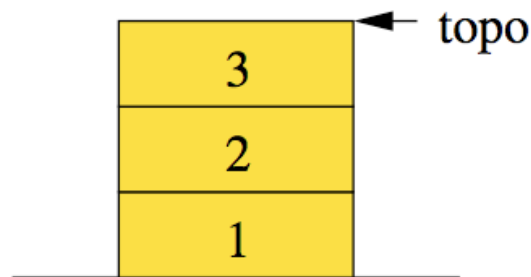
Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães

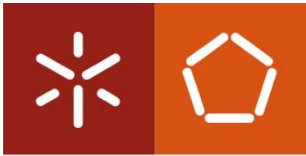


Universidade do Minho

Pilhas (Stacks)

- Estruturas eficientes para armazenar e retirar dados na ordem LIFO - LAST IN FIRST OUT (último a entrar, primeiro a sair);
- Aplicações: chamada de funções, recursividade, calculadoras, ...
- Estrutura de dados linear em que inserção e a remoção de elementos se faz pela mesma extremidade, designada por topo da pilha
- Uma pilha pode ser considerada como uma *lista*, mas com certas regras de restrição.
- Como é uma estrutura de dados mais simples que a lista, é possível obter implementações mais eficazes





Universidade do Minho

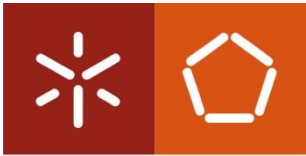
Pilhas (Stacks)

Manipulação (operações/métodos) sobre pilhas:

- peek - inspeciona qual o elemento no topo, não retira
- pop - remove elemento do topo
- push - inclui elemento no topo
- create – cria uma nova stack
- destroy – apaga uma stack
- empty - determina se a stack está vazia.

Implementação:

- Uma pilha pode ser implementada de diversas formas: vetor, vetor dinâmico, listas ligadas, etc...
- No exemplo a pilha é implementada através de uma lista ligada simples

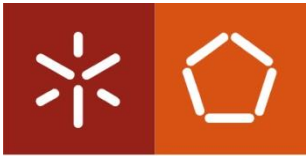


Universidade do Minho

Pilhas (Stacks)

- Exemplo:

```
/* definição da lista ligada */  
typedef struct node {  
    data_t data;  
    struct node* next;  
} node_t;  
/* definição da stack */  
typedef struct stack {  
    node_t *topo;  
} stack_t;
```

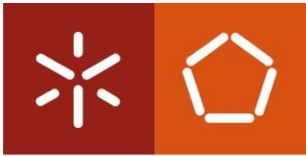


Universidade do Minho

Pilhas (Stacks)

- create
 - Uma possível implementação em C

```
stack_t * create(){  
    stack_t * aux=(stack_t *)malloc(sizeof(stack_t));  
    aux->topo=NULL;  
    return aux;  
}
```

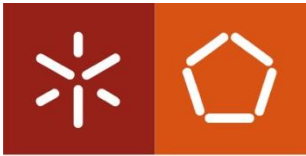


Universidade do Minho

Pilhas (Stacks)

- push
 - Uma possível implementação em C

```
void push(stack_t *s, data_t d){
node_t *elemento=(node_t *)malloc(sizeof(node_t));
elemento->data=d;
elemento->next=NULL;
if(s->topo == NULL){
s->topo = elemento;
}
else{
elemento->next =s->topo;
s->topo = elemento;
}}
```

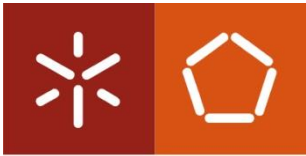


Universidade do Minho

Pilhas (Stacks)

- pop
 - Uma possível implementação em C

```
data_t* pop(stack_t *s){
    node_t *aux;
    data_t *p=NULL;
    aux=s->topo;
    if (s->topo != NULL){
        s->topo = aux->next;
        p=(data_t *)malloc(sizeof(data_t));
        *p=aux->data;
        free(aux);
        aux=NULL;
    }
    return p;
}
```

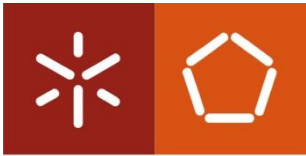


Universidade do Minho

Pilhas (Stacks)

- peek
 - Uma possível implementação em C

```
data_t* peek(stack_t *s){  
    data_t *p=NULL;  
    if (s->topo != NULL){  
        p=(data_t *)malloc(sizeof(data_t));  
        *p=s->topo->data;  
    }  
    return p;  
}
```

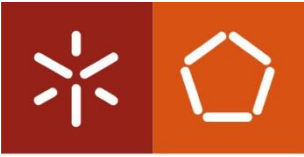



Universidade do Minho

Pilhas (Stacks)

- empty
 - Uma possível implementação em C

```
int empty(stack_t *s){  
    if (s->topo==NULL)  
        return 1;  
    else  
        return 0;  
}
```

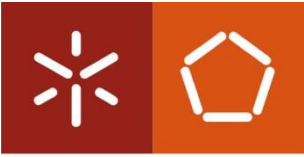


Universidade do Minho

Pilhas (Stacks)

- destroy
 - Uma possível implementação em C

```
void destroy(stack_t **s){
    node_t *aux=(*s)->topo;
    node_t *ant=aux;
    while (aux!=NULL)
    {
        aux=aux->next;
        free(ant);
        ant=aux;
    }
    free(*s);
    *s=NULL;
}
```

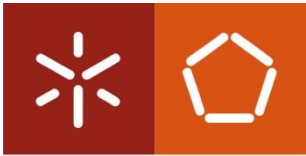


Universidade do Minho

Pilhas (Stacks)

- destroy
 - outra possível implementação em C

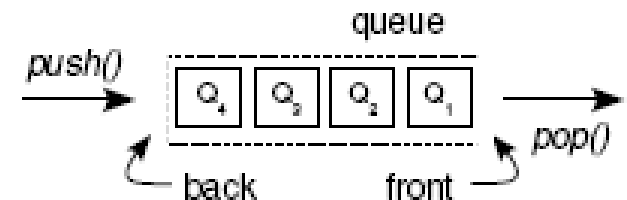
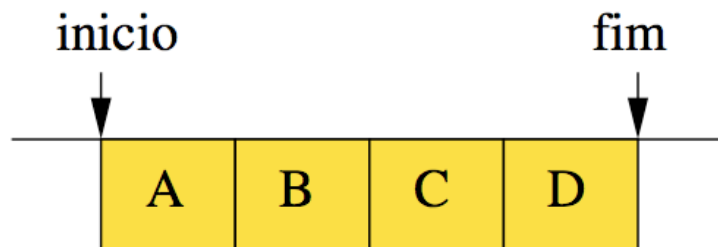
```
void destroy(stack_t **s){  
    while (!empty(*s))  
        pop(*s);  
    free(*s);  
    *s=NULL;  
}
```

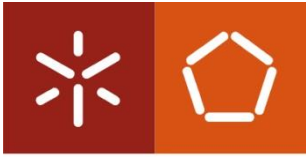


Filas (Queues)

Universidade do Minho

- Estruturas eficientes para armazenar e retirar dados na ordem FIFO - FIRST IN FIRST OUT (primeiro a entrar, primeiro a sair)
- Aplicações: buffers, simulação de filas de espera em supermercados, ...
- Estrutura de dados linear em que:
 - inserção de elementos se faz por uma extremidade, designada por cauda
 - remoção de elementos se faz por extremidade oposta à cauda, designada por cabeça
- uma fila pode ser considerada como uma *lista* “especial”





Filas (Queues)

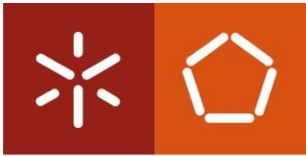
Universidade do Minho

Manipulação (operações/métodos) sobre filas:

- peek - inspeciona qual o elemento no início, não retira
- dequeue - remove elemento do início
- enqueue - inclui elemento no fim
- create – cria uma nova fila
- destroy – apaga uma fila
- empty - determina se a fila está vazia.

Implementação:

- Uma fila pode ser implementada de diversas formas: vetor, vetor dinâmico, listas ligadas, etc...

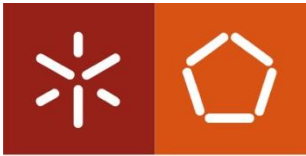


Universidade do Minho

Filas (Queues)

- Exemplo:

```
/* definição da lista ligada */  
typedef struct node {  
    data_t data;  
    struct node* next;  
} node_t;  
/* definição da queue */  
typedef struct queue {  
    node_t *head;  
    node_t *tail;  
} queue_t;
```

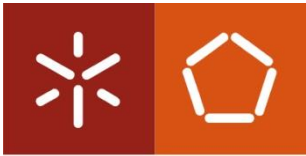


Universidade do Minho

Filas (Queues)

- create
 - Uma possível implementação em C

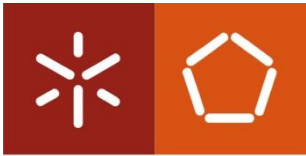
```
queue_t * create(){  
    queue_t * aux=(queue_t *)malloc(sizeof(queue_t));  
    aux->head=NULL;  
    aux->tail=NULL;  
    return aux;  
}
```



Filas (Queues)

- enqueue
 - Uma possível implementação em C
- ```
void enqueue(queue_t *q, data_t d){
 node_t *elemento=(node_t *)malloc(sizeof(node_t));
 elemento->data=d;
 elemento->next=NULL;
 if(q->head == NULL){
 q->head = elemento;
 q->tail=elemento;
 }
 else{
 (q->tail)->next = elemento;
 q->tail=elemento;
 }
}
```



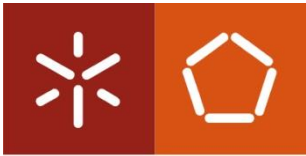


Universidade do Minho

# Filas (Queues)

- dequeue
  - Uma possível implementação em C

```
data_t* dequeue(queue_t *q){
 node_t *aux;
 data_t *p=NULL;
 aux=q->head;
 if (q->head != NULL){
 q->head = aux->next;
 p=(data_t *)malloc(sizeof(data_t));
 *p=aux->data;
 free(aux);
 aux=NULL;
 }
 return p;
}
```

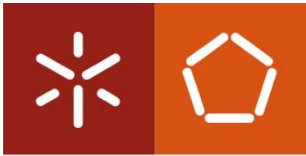


Universidade do Minho

# Filas (Queues)

- peek
  - Uma possível implementação em C

```
data_t* peek(queue_t *q){
 data_t *p=NULL;
 if (q->head != NULL){
 p=(data_t *)malloc(sizeof(data_t));
 *p=q->head->data;
 }
 return p;
}
```

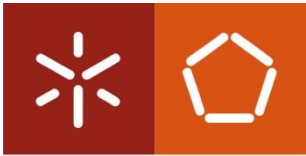


Universidade do Minho

# Filas (Queues)

- empty
  - Uma possível implementação em C

```
int empty(queue_t *q){
 if (q->head==NULL)
 return 1;
 else
 return 0;
}
```

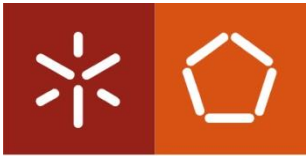


Universidade do Minho

# Filas (Queues)

- destroy
  - Uma possível implementação em C

```
void destroy(queue_t **q){
 node_t *aux=(*q)->head;
 node_t *ant=aux;
 while (aux!=NULL)
 {
 aux=aux->next;
 free(ant);
 ant=aux;
 }
 free(*q);
 *q=NULL;
}
```



Universidade do Minho

# Filas (Queues)

- destroy
  - outra possível implementação em C

```
void destroy(queue_t **q){
 while (!empty(*q))
 dequeue(*q);
 free(*q);
 *q=NULL;
}
```